Assumption Engine Developer Build Guide

Step-by-step: what to build, in what order, with the exact formulas, data formats, API contracts, a fully traced numeric example and test vectors you can check your code against

12 BUILD STEPS | CONFIG SCHEMAS | ALGORITHM | WORKED TRACE | API | TESTS

Reference implementation (Python, standard library only) is in assumption_engine/ in the repository.

You may build in any language. Your results must match the test vectors in Section 12.

Version 0.2 | 14 November 2026 | Replaces the earlier 'Flow and Logic' overview for implementation work

START HERE

What you are building, in one page

You are building a small **calculation service**. A caller says: "*Saudi Arabia, E-Commerce, B2B enterprise deal, strategic partnership channel*". The service answers with the numbers a financial model needs for that offline channel: how many leads one partner brings per month, what share qualify, become opportunities and are won, how many days a deal takes, and a low and high range. A second function turns a monthly activity plan into a monthly forecast of leads and deals.

	You send	You get back
Request	country, industry, deal_type, channel	—
Example	"Saudi Arabia", "E-Commerce", "B2B_ENT", "PAR_STRATEGIC"	—
Response (rounded)	—	leads/partner-month 2.25, qualified 61%, opportunity 49%, win 35%, cycle 180 days, deals per partner-month 0.241 (range 0.149 to 0.368)

The whole system is three layers

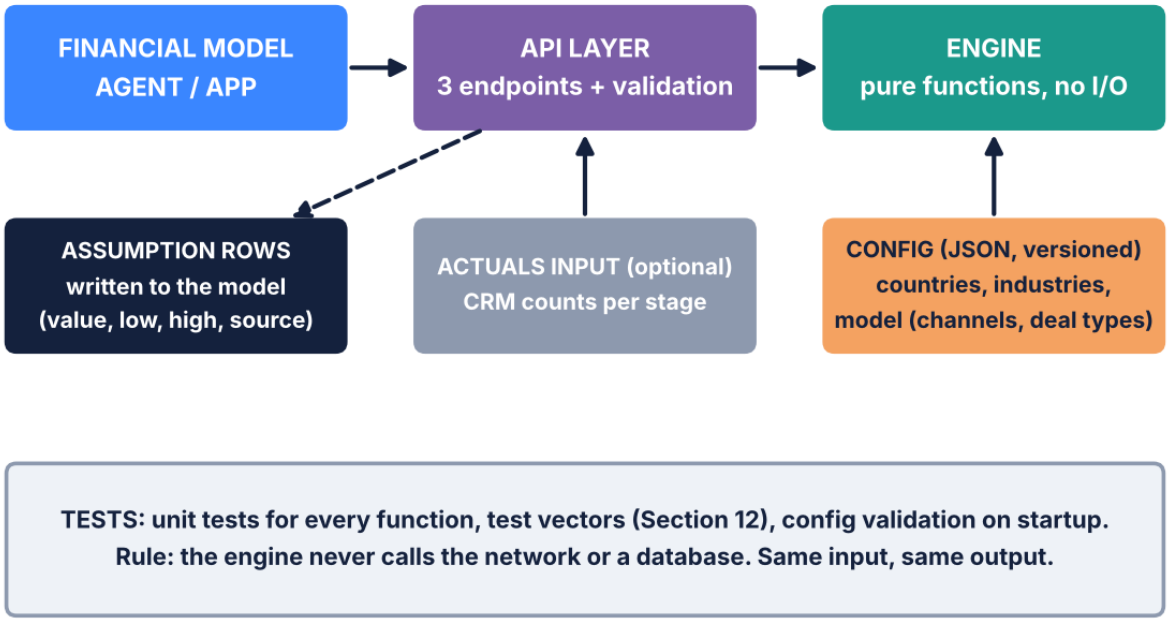


Figure 1. Three layers: config (data), engine (math), API (inputs, validation, outputs).

The 12 build steps (each has its own section with a definition of done)

#	Build step	You are done when
1	Repository and config files	The three JSON files load and pass the validation rules.
2	Request and response schemas	Bad input returns a clear 422 error list.
3	Name normalisation	'Uae', 'KSA', 'ecommerce' resolve; unknown names fail.
4	Combination check and country gate	Iran returns REVIEW_REQUIRED; B2C + cold BD returns NOT_APPLICABLE.
5	Core calculation (z, shifts, funnel)	Test vector T1 matches to 4 decimals.
6	Uncertainty range	P10 below base below P90 for every output.
7	Blend with company actuals	Test vector T11 matches; provenance changes.
8	Seasonality and partner ramp	Test vector T12 matches.
9	Forecast with sales-cycle lag	Total deals conserved; T13 matches.
10	Output rows for the financial model	Row schema validated; every row has provenance.
11	API endpoints and agent tools	Three endpoints return the documented JSON.
12	Tests and acceptance	All 13 vectors, property tests and the full-matrix check pass.

STEP 1

Repository and config files

Goal: Put all business knowledge in data files so analysts can change ratings without a code release.

Suggested layout (the reference implementation uses exactly this):

```
assumption_engine/
  config/
    countries.json      # 20 countries: ratings, tier, gate, ramadan sensitivity, aliases
    industries.json     # 28 industries: ratings, aliases
    model.json          # channels, deal types, uncertainty, seasonality, blend constants
  data/
    countries_list.csv  # the two lists supplied by the business (source of truth for names)
    industries_list.csv
    matrix_base.csv     # generated: every valid combination (14,000 rows: 10,500 numeric + 3,500 status)
  engine.py             # pure functions: assume(), forecast(), season()
  tests/test_engine.py
```

countries.json (shape and meaning)

```
{ "factor_defs": { "rel": "Relationship dependence", "dig": "...", "spd": "...", "evt": "...", "par": "...", "mkt": "...", "gov": "...
  "countries": {
    "Saudi Arabia": { "rel":5,"dig":3,"spd":3,"evt":5,"par":4,"mkt":5,"gov":5,"fric":2,"loc":4,
                      "tier":"A", "gate":"normal", "ram":1.0 },
    "Iran":         { ..., "tier":"D", "gate":"review_required", "ram":0.9 } },
  "aliases": { "uae":"UAE", "ksa":"Saudi Arabia", "turkiye":"Turkey", ... } }
```

Field	Type / allowed values	Meaning
rel, dig, spd, evt, par, mkt, gov, fric, loc	integer 1 to 5	Factor ratings. 3 is neutral. For fric, higher is worse.
tier	A, B, C, D	Sets uncertainty width (Step 6).
gate	normal | elevated | review_required	Controls Step 4 behaviour.
ram	number 0 to 1	Share of the Ramadan effect that applies in this country (Step 8).
aliases	lower-case string to canonical name	Used in Step 3.

industries.json

```
{ "factor_defs": { "cyc":"Sales-cycle length", "acv":"Typical ticket size", "par":"Partner dependence", "evt":"Event dependence", "reg"
  "industries": { "E-Commerce": { "cyc":3,"acv":3,"par":4,"evt":4,"reg":2,"cmt":3 }, ... 28 entries ... },
  "aliases": { "ecommerce":"E-Commerce", "proptech":"PropTech & ConTech", ... } }
```

model.json, the channel entry (the most important structure)

```
"channels": { "PAR_STRATEGIC": {
   "name": "Partnership: strategic / co-sell / integration",
   "unit": "active_partner_month",
   "lead": 1.7,   "q": 0.60,  "o": 0.45,  "w": 0.30,   "cyc": 180,  "ramp": 6,
   "V": { "c_par": 0.10, "i_par": 0.10, "c_gov": 0.04 },              # leads per unit
   "Q": { "c_par": 0.05, "i_acv": 0.04 },                             # qualified rate
   "O": { "c_par": 0.05, "c_rel": 0.06, "i_cmt": -0.06 },             # opportunity rate
   "W": { "c_rel": 0.06, "c_fric": -0.07, "i_reg": -0.06, "i_cmt": -0.05 },  # win rate
   "T": { "i_cyc": 0.14, "i_cmt": 0.08, "c_spd": -0.08 } } }          # cycle days
```

Key	Meaning
lead, q, o, w, cyc	Base funnel for a neutral market (all ratings 3) and deal type B2B_ENT. Leads per unit; three rates between 0 and 1; cycle in days.
ramp	Months for a new partner to reach full productivity. 0 for events and BD.
V, Q, O, W, T	Sensitivity maps. Key = factor name (c_ = country, i_ = industry). Value = coefficient per rating step.
deal_types	Per deal type: V (leads multiplier), q, o, w (logit shifts), T (cycle multiplier), channels (all, or a list), gov_v (extra volume effect from state share of demand).

Config validation (run at startup; refuse to start on failure)

- Every country has all 9 factors and every industry all 6, each an integer from 1 to 5.
- Every key used in V, Q, O, W, T exists as c_xxx or i_xxx in the factor lists.
- Base rates q, o, w are strictly between 0 and 1. lead, cyc are greater than 0.

- Every name in data/countries_list.csv and data/industries_list.csv resolves to a config entry (the repository test does this).
- gate is one of the three allowed values; tier is A, B, C or D.

STEP 2

Request and response schemas

Goal: Define exactly what goes in and out so the agent and the service never disagree.

Request for one combination

```
POST /v1/assumptions
{ "country": "Saudi Arabia",           // required, any alias accepted
  "industry": "E-Commerce",            // required
  "deal_type": "B2B_ENT",              // required: B2B_SMB | B2B_ENT | B2G | B2C
  "channel": "PAR_STRATEGIC",          // required: EVT_EXHIBIT | EVT_HOSTED | PAR_RESELLER | PAR_STRATEGIC | PAR_REFERRAL | BD_COLD |
  "actuals": {                         // optional: company CRM data (Step 7)
     "qualified_rate":   { "successes": 12, "trials": 40 },
     "opportunity_rate": { "successes": 6,  "trials": 12 },
     "win_rate":         { "successes": 2,  "trials": 40 },
     "leads_per_unit":   { "leads": 30, "units": 30 },
     "cycle_days":       { "mean": 130, "n": 12 } } }
```

Response when status is OK

```
{ "status": "OK", "country": "Saudi Arabia", "industry": "E-Commerce", "deal_type": "B2B_ENT", "channel": "PAR_STRATEGIC",
  "unit": "active_partner_month", "provenance": "prior", "sigma": 0.269, "warnings": [], "ramp_months": 6,
  "leads_per_unit":   {"base": 2.249, "p10": 1.593, "p90": 3.176},
  "qualified_rate":   {"base": 0.612, "p10": 0.528, "p90": 0.69},
  "opportunity_rate": {"base": 0.492, "p10": 0.407, "p90": 0.578},
  "win_rate":         {"base": 0.355, "p10": 0.28, "p90": 0.437},
  "cycle_days":       {"base": 180.0, "p10": 127.5, "p90": 254.2},
  "deals_per_unit":   {"base": 0.2405, "p10": 0.1494, "p90": 0.3679},
  "units_per_deal": 4.16, "leads_per_deal": 9.35,
  "drivers": { "V": [...top 3...], "Q": [...], "O": [...], "W": [{"factor": "Country: Relationship dependence", "rating": 5, "effect":
```

Statuses and errors

Status / HTTP	When	Body
OK / 200	Combination valid, country not review_required.	Full response above.
NOT_APPLICABLE / 200	Deal type does not allow this channel (B2C with BD_COLD or BD_WARM, or with PAR_STRATEGIC).	status, warnings[0] explains. No numbers.
REVIEW_REQUIRED / 200	Country gate is review_required (Libya, Iran, Syria, Sudan, Yemen).	status, warnings[0] says compliance review is needed. No numbers.
VALIDATION_ERROR / 422	Unknown country, industry, deal type or channel; wrong types; rating out of range in actuals.	{"errors":[{"field":"country","message":"Unknown country: Atlantis","supported":[...]}]}
INTERNAL / 500	Config invalid or bug.	Log and alert. Never return partial numbers.

Why REVIEW_REQUIRED is 200 and not an error: it is a correct business answer ("we will not estimate this"), not a failure of the request. The agent must read the status field every time.

STEP 3

Name normalisation

Goal: Accept the way people really type names, but never guess.

```
function normalise(name, table, aliases):
    key = trim(name)
    for each canonical in table:                # exact match ignoring case
        if lower(canonical) == lower(key): return canonical
    if lower(key) in aliases: return aliases[lower(key)]
    raise ValidationError("Unknown ...", supported = keys(table))
```

Input	Output	Why
"Uae" (as written in your countries file)	UAE	case-insensitive match
"KSA"	Saudi Arabia	alias
"turkiye"	Turkey	alias; the list uses 'Turkey'
"ecommerce"	E-Commerce	alias
"proptech"	PropTech & ConTech	alias
"Atlantis"	error 422	unknown; return the supported list so the agent can ask the user

STEP 4

Combination check and country gate

Goal: Stop early, with a clear status, when a request must not produce numbers.

```
# order matters: check in exactly this order
1. if deal_type not in deal_types      -> 422
2. if channel   not in channels        -> 422
3. allowed = deal_types[deal_type].channels
   if allowed != "all" and channel not in allowed:
        return { status: "NOT_APPLICABLE", warnings: ["B2C is not modelled for channel BD_COLD."] }
4. gate = countries[country].gate
   if gate == "review_required":
        return { status: "REVIEW_REQUIRED", warnings: ["<country>: compliance/operability review required ..."] }
5. if gate == "elevated": warnings.append("<country>: elevated operating risk; treat ranges as wide ...")
   continue to Step 5
```

Deal type	Channels allowed
B2B_SMB, B2B_ENT, B2G	all seven
B2C	EVT_EXHIBIT, EVT_HOSTED, PAR_REFERRAL, PAR_RESELLER

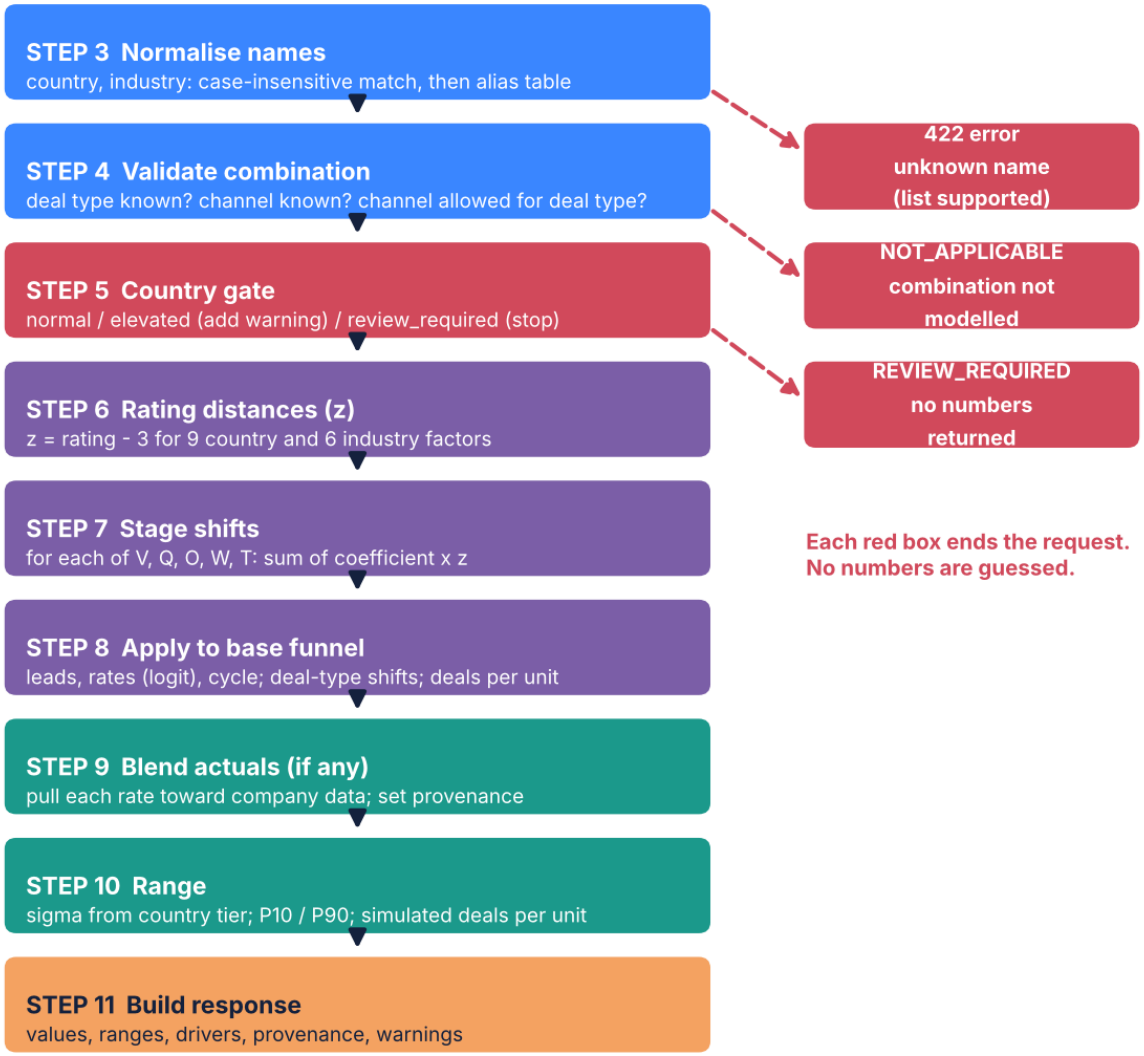


Figure 2. Request flow for one combination. Numbers match the step numbers in this guide. Red boxes end the request.

STEP 5

Core calculation: ratings to funnel numbers

Goal: Turn the 15 ratings into five shifts, then into the funnel. This is the heart of the engine.

5.1 Rating distance (z)

For every country factor k and industry factor k: `z[c_k] = country rating - 3`, `z[i_k] = industry rating - 3`. Range -2 to +2. Factors with z = 0 contribute nothing and are skipped.

5.2 Stage shifts

The channel has five sensitivity maps V, Q, O, W, T. For each stage: `shift[stage] = sum over (factor, coefficient) in map of coefficient x z[factor]`. Keep each product: you will need the three largest as the 'drivers' in the response.

5.3 Funnel

```
if deal_type.gov_v != 0: shift.V += deal_type.gov_v * z[c_gov]      # B2G only. Do this BEFORE computing lead.
lead  = channel.lead * deal_type.V * exp(shift.V)
q     = sigmoid( logit(channel.q) + deal_type.q + shift.Q )
o     = sigmoid( logit(channel.o) + deal_type.o + shift.O )
w     = sigmoid( logit(channel.w) + deal_type.w + shift.W )
cycle = channel.cyc * deal_type.T * exp(shift.T)
deals_per_unit = lead * q * o * w
units_per_deal = 1 / deals_per_unit
leads_per_deal = 1 / (q * o * w)

logit(p)   = ln( p / (1 - p) )          sigmoid(x) = 1 / (1 + e^(-x))
```

Two details that cause most bugs

1) Rates must use logit shifts, never add percentage points: it keeps every rate strictly between 0 and 1. 2) For B2G, add the gov_v term to shift.V **before** computing leads, as in the first line above.

WORKED TRACE

One full calculation, every number

Input: **Saudi Arabia, E-Commerce, B2B_ENT, PAR_STRATEGIC**. Reproduce every line below. If your numbers differ, find the first line that differs.

A. Ratings and distances (only non-zero z shown)

Factor	Rating	z
Country: Relationship dependence	5	+2
Country: Event ecosystem strength	5	+2
Country: Partner / channel ecosystem maturity	4	+1
Country: Reachable buyer density	5	+2
Country: State share of demand	5	+2
Country: Payment / contract / FX friction	2	-1
Country: Local-language need	4	+1
Industry: Partner dependence	4	+1
Industry: Event dependence	4	+1
Industry: Regulation / approvals exposure	2	-1

B. Stage shifts (coefficient x z)

Stage	Terms (coefficient x z = product)	Shift
V leads	Country: Partner / channel ecosystem maturity: +0.10 x +1 = +0.10 Industry: Partner dependence: +0.10 x +1 = +0.10 Country: State share of demand: +0.04 x +2 = +0.08	+0.280
Q qualified	Country: Partner / channel ecosystem maturity: +0.05 x +1 = +0.05	+0.050
O opportunity	Country: Partner / channel ecosystem maturity: +0.05 x +1 = +0.05 Country: Relationship dependence: +0.06 x +2 = +0.12	+0.170
W win	Country: Relationship dependence: +0.06 x +2 = +0.12 Country: Payment / contract / FX friction: -0.07 x -1 = +0.07 Industry: Regulation / approvals exposure: -0.06 x -1 = +0.06	+0.250
T cycle	none	+0.000

C. Apply to the base funnel (PAR_STRATEGIC base: lead 1.7, q 0.60, o 0.45, w 0.30, cycle 180; deal type B2B_ENT has no shifts)

Quantity	Calculation	Result
leads per partner-month	1.7 x 1.0 x exp(+0.280)	2.2493
qualified rate	sigmoid(logit(0.6) +0.0 +0.050) = sigmoid(+0.4055 +0.050)	0.6119
opportunity rate	sigmoid(logit(0.45) +0.0 +0.170) = sigmoid(-0.2007 +0.170)	0.4923
win rate	sigmoid(logit(0.3) +0.0 +0.250) = sigmoid(-0.8473 +0.250)	0.3550
cycle days	180 x 1.0 x exp(+0.000)	180.00
deals per partner-month	2.2493 x 0.6119 x 0.4923 x 0.3550	0.2405
partner-months per deal	1 / 0.2405	4.16

Compare: Egypt with the same inputs gives leads 2.1611, win 0.3106, cycle 195.0, deals per partner-month 0.1961 (test vector T2).

D. Drivers to return (three largest |coefficient x z| per stage)

Sort the products of each stage by absolute value, keep the top three. For leads and cycle report the effect as a percentage: `(exp(product) - 1) x 100`. For rates report the logit shift. Example, win rate: Country: Relationship dependence (rating 5) +0.12; Country: Payment / contract / FX friction (rating 2) +0.07; Industry: Regulation / approvals exposure (rating 2) +0.06.

STEP 6

Uncertainty range (P10 and P90)

Goal: Never return a single number. Wider range where we know less.

```
sigma_country = { A: 0.25, B: 0.35, C: 0.45, D: 0.60 }[ country.tier ]
sigma_industry = 0.10
s0 = sqrt( sigma_country^2 + sigma_industry^2 )                 # e.g. tier A: sqrt(0.0625 + 0.01) = 0.2693
z90 = 1.2816                                                    # 90th percentile of standard normal

for a rate p:        P10 = sigmoid( logit(p) - z90 * s0 )       P90 = sigmoid( logit(p) + z90 * s0 )
for leads, cycle v:  P10 = v * exp( -z90 * s0 )                 P90 = v * exp( +z90 * s0 )

deals_per_unit range (simulation, 4000 draws, fixed seed 7):
    each draw: lead*exp(N(0,s0)) * sigmoid(logit(q)+N(0,s0)) * sigmoid(logit(o)+N(0,s0)) * sigmoid(logit(w)+N(0,s0))
    sort draws; P10 = draw[ int(0.10*4000) ], P90 = draw[ int(0.90*4000) ]
```

Tier	Countries	sigma_country	s0 (with industry 0.10)
A	Saudi Arabia, UAE, Qatar, Kuwait, Bahrain, Oman	0.25	0.2693
B	Egypt, Jordan, Morocco, Tunisia, Turkey	0.35	0.3640
C	Lebanon, Algeria, Iraq, Palestine	0.45	0.4610
D	Libya, Iran, Syria, Sudan, Yemen (gated; no numbers)	0.60	0.6083

Cross-language note: rate and volume ranges are closed-form, so your results must match exactly. The simulated deals-per-unit range depends on the random generator; match it within 5% (more draws if you want a tighter match), and keep the seed fixed so results are repeatable inside your own system.

STEP 7

Blend with company actuals

Goal: As the company records real results, move away from the prior toward the data and say so.

```
pseudo-counts: rate = 15,  volume_units = 3,  cycle_n = 5

for each rate stage (q, o, w) with actual {successes, trials}:
      rate = ( prior_rate * 15 + successes ) / ( 15 + trials )
leads_per_unit with {leads, units}:  lead  = ( prior_lead * 3 + leads ) / ( 3 + units )
cycle_days with {mean, n}:           cycle = ( prior_cycle * 5 + mean * n ) / ( 5 + n )

n_eff = max( trials of each rate, 5 * units, n of cycle )
provenance = "company_actuals" if n_eff >= 50 else "blended"       # "prior" when no actuals
s0 = s0 / sqrt( 1 + n_eff / 15 )                                   # narrower range with more data
```

Worked example (test vector T11): Egypt, SaaS, B2B_ENT, BD_WARM; the company actually won 2 of 40 opportunities.

Quantity	Calculation	Result
Prior win rate	from Step 5	0.3127
Blended win rate	(0.3127 x 15 + 2) / (15 + 40)	0.1217
n_eff, provenance	n_eff = 40, below 50	blended
s0 before	tier B: sqrt(0.35^2 + 0.10^2)	0.3640
s0 after	0.3640 / sqrt(1 + 40/15)	0.1900

Only the stages that have actuals are blended; the others stay at their prior. All blending uses the final prior values from Step 5 (including deal-type shifts).

STEP 8

Seasonality and partner ramp-up

Goal: Business activity dips in Ramadan, Eid and summer; new partners take months to become productive.

```
season(country, year, month):
    ram = countries[country].ram ;  days = number of days in the month ;  volume = 1.0 ;  cycle = 1.0
    # Ramadan window for that year from model.json (approximate dates). overlap = days of the month inside the window / days
    volume *= 1 - overlap * ram * (1 - 0.65)
    cycle  *= 1 + overlap * ram * (1.25 - 1)
    # Eid al-Adha: window = start date + 4 days (5 days). overlap computed the same way
    volume *= 1 - overlap_eid * ram * (1 - 0.50)
    # Summer: if month in [7, 8]:  volume *= 0.85 ;  cycle *= 1.10
    return { volume, cycle }
```

Worked example (test vector T12): Saudi Arabia, February 2027

Step	Calculation	Result
Ramadan window 2027	8 Feb to 9 Mar (model.json)	—
Days of February inside window	8 Feb to 28 Feb	21 of 28 days
overlap	21 / 28	0.7500
volume multiplier	1 - 0.75 x 1.0 x (1 - 0.65)	0.7375
cycle multiplier	1 + 0.75 x 1.0 x (1.25 - 1)	1.1875
October 2027 (no events)	volume and cycle multipliers	1.0000 and 1.0000

Partner ramp-up

```
if channel.ramp > 0 and row.partner_age_months is provided:
      ramp_factor = min( 1.0, partner_age_months / channel.ramp )      # reseller 4, strategic 6, referral 2
else: ramp_factor = 1.0
effective_units = row.units * ramp_factor
```

Example: a strategic partner (ramp 6) that signed 3 months ago produces at 3 / 6 = 50% of full productivity.

STEP 9

Forecast with sales-cycle lag

Goal: Convert a monthly activity plan into monthly leads, opportunities and deals, with deals landing after the cycle time.

```
plan row: { country, industry, deal_type, channel, month "YYYY-MM", units, partner_age_months? }
output row per month: { month, leads, qualified, opps, deals, deals_p10, deals_p90, revenue_base? }

for each row:
    a  = assume(country, industry, deal_type, channel)          # Steps 3-6; stop if status != OK
    s  = season(a.country, year, month) ;  u = units * ramp_factor
    leads = u * a.leads_per_unit * s.volume ;  qualified = leads * a.q ;  opps = qualified * a.o ;  deals = opps * a.w
    add leads, qualified, opps to the row's own month
    cycle_months = a.cycle_days * s.cycle / 30
    for (multiplier, weight) in [(0.6,.15), (0.8,.25), (1.0,.30), (1.2,.20), (1.5,.10)]:
         lag = cycle_months * multiplier ;  lo = floor(lag) ;  frac = lag - lo
         add deals * weight * (1 - frac) to month (row month + lo)        # split across two months
         add deals * weight * frac       to month (row month + lo + 1)    # (skip a part whose share is 0)
         add u*s.volume*a.deals_per_unit.p10 * weight * share to deals_p10 (same months); same for p90
if acv given: revenue_base = deals * acv
```

Worked example (test vector T13): UAE, SaaS, B2B_ENT, BD_WARM, 20 meetings in April 2027

Per-unit values: cycle 92.31 days (3.077 months), total deals from 20 meetings = 1.8386.

Multiplier	Weight	Lag in months	Whole months + fraction	Lands in
0.6	15%	1.846	1 + 0.846	2027-05 (15%), 2027-06 (85%)
0.8	25%	2.462	2 + 0.462	2027-06 (54%), 2027-07 (46%)
1.0	30%	3.077	3 + 0.077	2027-07 (92%), 2027-08 (8%)
1.2	20%	3.692	3 + 0.692	2027-07 (31%), 2027-08 (69%)
1.5	10%	4.616	4 + 0.616	2027-08 (38%), 2027-09 (62%)

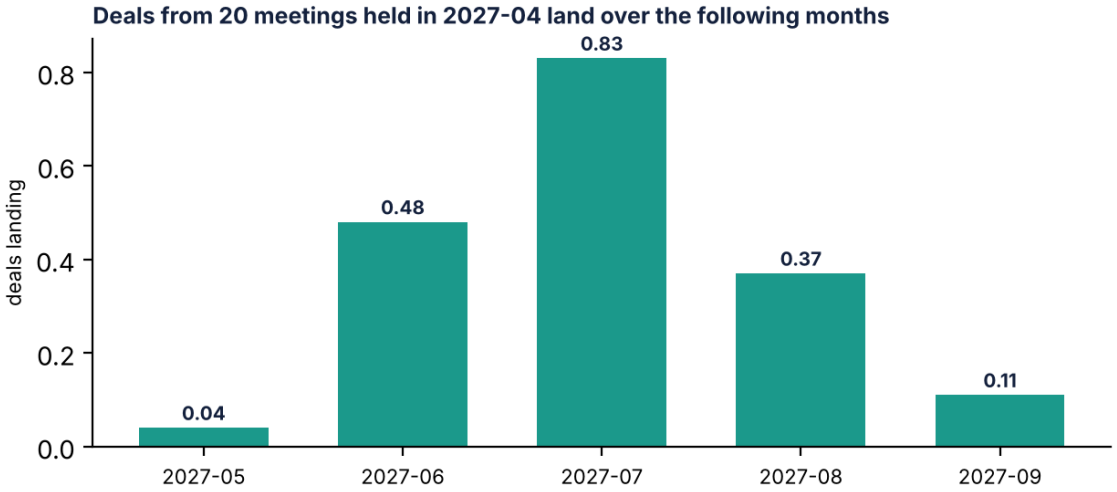


Figure 3. Monthly deal landing for the example. The bars add up to 1.839, the total from Step 5, so no deals are created or lost. Your test must check this.

STEP 10

Output rows for the financial model

Goal: Give the model one flat row per assumption so it can load, show and audit them.

```
{
 "key": "ksa.ecommerce.b2b_ent.par_strategic.win_rate",
 "value": 0.355,
 "low_p10": 0.28,
 "high_p90": 0.437,
 "unit": "share of opportunities won",
 "channel": "PAR_STRATEGIC",
 "provenance": "prior",
 "methodology": "assumption_engine v0.2",
 "drivers": [
  "Country: Relationship dependence",
  "Country: Payment / contract / FX friction",
  "Industry: Regulation / approvals exposure"
 ],
 "warnings": [],
 "review_required": false
}
```

Build one row for each of: leads_per_unit, qualified_rate, opportunity_rate, win_rate, cycle_days, deals_per_unit. Key format: country.industry.deal_type.channel.metric in lower case with underscores.

Field	Rule
provenance	prior, blended or company_actuals. Never empty.
review_required	true when provenance is prior AND the channel is a large share of planned pipeline, or the country gate is elevated, or any factor was overridden by a user. The agent sets the pipeline-share rule; the engine sets the other two.
warnings	Copy from the response. Show them to the user.

STEP 11

API endpoints and agent tools

Goal: Expose three operations. The agent calls them as tools.

Endpoint	Purpose	Request	Response
GET /v1/supported	List what the engine supports	none	countries[21], industries[29], deal_types[4], channels[7] (with units), aliases
POST /v1/assumptions	Assumptions for one combination (Steps 3 to 10)	Section 'Step 2' body	status and numbers or error
POST /v1/forecast	Monthly forecast for a plan (Step 9)	{ "plan":[...rows...], "acv": 50000 }	{ "months":[{month, leads, qualified, opps, deals, deals_p10, deals_p90, revenue_base}] }

Agent tool definition (function-calling format)

```
{ "name": "get_channel_assumptions",
  "description": "Funnel assumptions for an offline channel by country, industry and deal type. Returns status OK, NOT_APPLICABLE or RE
  "parameters": { "type": "object", "required": ["country","industry","deal_type","channel"],
    "properties": {
      "country":   { "type": "string" },
      "industry":  { "type": "string" },
      "deal_type": { "type": "string", "enum": ["B2B_SMB","B2B_ENT","B2G","B2C"] },
      "channel":   { "type": "string", "enum": ["EVT_EXHIBIT","EVT_HOSTED","PAR_RESELLER","PAR_STRATEGIC","PAR_REFERRAL","BD_COLD","BD_
      "actuals":   { "type": "object" } } } }
```

Agent behaviour rules (put in the system prompt)

- Call GET /v1/supported first and use only those names. If the user's wording is unknown, ask which supported value they mean.
- Always read `status`. If REVIEW_REQUIRED or NOT_APPLICABLE, tell the user and stop. Do not estimate from another country.
- Report provenance in plain words, and always show the range with the base value.
- Ask for missing inputs: planned units per channel and month, ACV, unit costs, any CRM actuals, deal type.

- Explain the top driver in one sentence. Set review_required on rows as described in Step 10.

STEP 12

Tests and acceptance

Goal: Prove your build matches the reference. All numbers below were produced by the reference engine.

ID	country / industry / deal / channel	leads	qual.	opp.	win	cycle	deals/unit	s0
T1	Saudi Arabia / E-Commerce / B2B_ENT / PAR_STRATEGIC	2.2493	0.6119	0.4923	0.3550	180.00	0.2405	0.2690
T2	Egypt / E-Commerce / B2B_ENT / PAR_STRATEGIC	2.1611	0.6119	0.4773	0.3106	194.99	0.1961	0.3640
T3	UAE / SaaS / B2B_SMB / BD_WARM	1.4512	0.4998	0.5073	0.4068	55.39	0.1497	0.2690
T4	Egypt / FinTech / B2G / EVT_EXHIBIT	67.8751	0.2673	0.2695	0.1317	397.53	0.6436	0.3640
T5	Turkey / Gaming / B2C / EVT_HOSTED	85.4121	0.4662	0.5451	0.1627	22.50	3.5305	0.3640
T6	Qatar / AI / B2B_ENT / BD_COLD	0.8353	0.1756	0.3145	0.1365	190.69	0.0063	0.2690
T7	Iraq / Logistics / B2B_ENT / PAR_RESELLER	2.4261	0.5000	0.4072	0.2144	172.54	0.1059	0.4610

Tolerance: 0.0001 on every column (rates and leads exact; cycle to 0.01). Deals/unit P10 and P90 are simulated: check within 5%.

ID	Input	Expected result
T8	Iran / AI / B2B_ENT / BD_WARM	status REVIEW_REQUIRED; no numeric fields; warning mentions compliance review
T9	UAE / Gaming / B2C / BD_COLD	status NOT_APPLICABLE; warning says B2C is not modelled for BD_COLD
T7b	Iraq (from T7)	status OK and exactly one warning about elevated operating risk
T10	"Uae" / "ecommerce" / B2B_ENT / BD_WARM	country normalised to UAE, industry to E-Commerce; same numbers as the canonical names
T11	Egypt / SaaS / B2B_ENT / BD_WARM with win_rate actuals 2 of 40	win 0.1217 (prior 0.3127); provenance blended; s0 0.1900
T12	season(Saudi Arabia, 2027, 2) and (Saudi Arabia, 2027, 10)	volume 0.7375, cycle 1.1875 and volume 1.0000, cycle 1.0000
T13	forecast: UAE SaaS B2B_ENT BD_WARM, 20 units, 2027-04	sum of deals = 1.8386 (conservation); deals appear in 2027-05, 2027-06, 2027-07, 2027-08, 2027-09

Property tests (run on all valid combinations)

- Every rate satisfies 0 < P10 <= base <= P90 < 1. Leads and cycle are positive.
- For OK results, deals_per_unit.base equals leads x qualified x opportunity x win to 1e-9.
- Calling the same input twice returns identical output (determinism).
- Increasing the friction rating of a country never increases its win rate; increasing relationship dependence never increases cold-BD qualified rate (monotonic checks).
- Forecast conserves deals: the sum over months equals units x ramp x season volume x deals_per_unit.
- Every name in data/countries_list.csv and data/industries_list.csv resolves.

Full-matrix check

Generate every valid combination and compare with **data/matrix_base.csv**: 20 countries x 28 industries x 25 valid deal-type and channel pairs = 14,000 rows. The 15 countries that are not gated give 10,500 rows with numbers; the 5 gated countries (Libya, Iran, Syria, Sudan, Yemen) give 3,500 rows whose status is REVIEW_REQUIRED and whose number columns are empty. Compare leads, qualified, opportunity, win and cycle using the rounding in the file. Skip the deals/unit P10 and P90 columns: the file uses only 300 draws.

Why 25 pairs: B2B_SMB, B2B_ENT and B2G allow all 7 channels (21 pairs); B2C allows 4 channels.

Definition of done for the whole build

- Test vectors T1 to T13 pass. Property tests pass. Config validation passes and fails correctly when a rating is set to 6.
- Changing a rating in countries.json changes the output without any code change.

- Agent tools return the documented JSON; an unknown country returns the 422 error with the supported list.
- Every response shows provenance; no response ever contains numbers for a REVIEW_REQUIRED or NOT_APPLICABLE status.

APPENDIX A

Edge cases and how to change the engine

Situation	Required behaviour
Rate would reach exactly 0 or 1	Cannot happen with logit shifts. If an input rate in config is 0 or 1, config validation must fail.
deals_per_unit is 0 or non-finite	Treat as a bug: return 500, never a number.
Month string not YYYY-MM, or year without a Ramadan window in config	Bad format: 422. Year without window: no Ramadan effect for that year, add a warning.
Plan row for a gated country	The whole forecast request fails with the status and reason of that row; do not drop the row silently.
partner_age_months missing for a partner channel	ramp_factor = 1 (full productivity). Add a warning saying ramp was not applied.
Actuals with trials = 0	Ignore that actual; treat as absent.
Empty plan	Return an empty months list.
Float precision	Compute in double precision; round only for display (leads 2 d.p., rates 4 d.p., cycle 1 d.p.).

How to change things (no code change unless noted)

I want to...	Do this
Change a country or industry rating	Edit the rating (1 to 5) in countries.json or industries.json. Restart. Re-run tests.
Open a gated country	Change gate to elevated or normal after compliance sign-off. Record who approved it and when.
Add a country or industry	Add the entry and its name to the CSV list; add aliases; run config validation.
Change how much a factor matters	Edit the coefficient in the channel's V, Q, O, W or T map.
Add a channel	Add a channel block in model.json (base funnel, ramp, five maps); add it to deal types that allow it. Code does not change.
Replace priors with real data	Send actuals (Step 7). When there are enough deals, fit new base rates and coefficients and update model.json. Keep a version log.
Use the local-language factor	Needs a decision: add 'c_loc' to the maps where it should matter. No code change.

What this engine does not do (do not build it)

- Costs of events or partners (taken from the financial model). Pricing or ACV (taken from the model). Currency conversion.
- Overlap between channels (a lead from both an event and a partner). Interactions between industry and country factors.
- Predicting any single deal. The output is an average expectation with a range.

Honest status of the numbers

All base funnels, the 15 ratings and the coefficients are expert priors written by the product team's assistant to get a working default. They are not measured from data. Your job is to implement the mechanics exactly; the business team's job is to review the priors and feed actuals. The service must label every output with its provenance so the difference is always visible.